The circuit bank — the plain version, with links to the real thing

Myrla Phillippe · October 2026 · every link opens the file on GitHub

Rules-only bank — 30 items, one difficulty; every circuit proposed by the enumerator (`source: enumerate`)	30-item bank folder	circuit-banks repo
LLM-proposed bank — 10 items; shapes proposed by the LLM fallback, checked by the critic (`source: llm`)	10-item bank folder	circuit-banks repo
The code, inside Chen's repo (fork, branch `circuit-bank`; private)	fork	the skill folder
The engine — circuit-diagram-skill (public)	circuit-diagram-skill	standalone circuit-bank-skill (private)

The problem

Distinction: this is the goal both versions share — it is Chen's definition, unchanged.

For a fair exam where every student gets a different question, you need a big pile of questions that are all “the same kind, same difficulty, but not the same.” Chen calls that an isomorphic bank: same objective, variation beyond numbers, a changed solution, same difficulty. Full definition and why it matters.

Chen's way

Distinction: this is what the method looked like before circuits — two AI agents and a counting loop.

An AI invents variations, another AI judges them, and a loop keeps count until you have enough. Numbers, pictures and answers were not done yet. Because the AI has no seed, the same input gives different variations each run — the saved sidecar file is what makes a bank reproducible, not regeneration.

Your way — rules first, the LLM only as a tagged fallback

Distinction: the same principle as your circuit skill's English front door — rules first; the AI only where no rule exists; and when it is used, it says so (every item carries `source: enumerate` or `source: llm`). Chen's loop is kept unchanged; his two AI agents become code; your circuit skill does every number.

Circuits are special: a computer can draw one, solve one, and tell if two are secretly the same. So:

1. List every possible circuit shape at one difficulty — there are only 34 three-step ones. No guessing. Where the LLM comes in: the list is exhaustive, so if you ask for more than exist the run stops and tells you to widen the box — an LLM cannot invent shapes that do not exist (tested: every fallback proposal came back as a duplicate). The LLM proposes only for families the enumerator does not cover (capacitors, switches, meters, other unknowns); step 2 checks those like any other, and each is tagged `source: llm`.
   → why 34, and what changing the box does

2. Check each with a script: does it solve? is it really new? right number of steps? values sensible?
   → what "values sensible" means · the four rejection reasons (report §4.4)
3. Solve and draw each one with your circuit skill — exact numbers, a figure, a worked solution.
   → circuit-diagram-skill
4. Make the wrong answers by deliberately making a common student mistake and computing what it gives.
   → worked example, q1 (report §2)
5. Package it in Chen's format, plus a PDF with an answer key.
   → the whole pipeline as one chart

What you got (30-item bank). 30 questions, all three-step, all different, all correct; the rules covered everything, so the fallback never fired — provenance: 30 enumerated, 0 from the LLM — and running it again gives the exact same 30.

The files: PHY2-DC-SPRTC3-10232026.yaml — the bank and its answer key · PHY2-DC-SPRTC3-10232026.pdf — worksheet + worked solutions · figures/

The LLM-proposed bank — what the fallback looks like (10 items)

Distinction: identical pipeline except step 1 — here the LLM proposed every circuit (it was the only proposer at the time). Everything after it is the same code, and every item is tagged source: llm.

1. Propose: an LLM writes candidate circuits (Chen's Agent 2, adapted to write circuit lingo). Different shapes every run; mixed difficulty (2–5 steps) because the difficulty rule did not exist yet.

2–5. The same script critic, the same circuit skill, the same computed distractors, the same packaging.

What you got. 10 correct, distinct items, reproducible from their sidecar — but not regenerable from the input alone, and not one difficulty. That is why the enumerator became the first choice and the LLM the fallback. Provenance: 0 enumerated, 10 from the LLM.

The files: PHY2-DC-SPRTC-10222026.yaml · PHY2-DC-SPRTC-10222026.pdf · figures/

The forked version — how it works inside Chen's repo

Distinction: nothing of Chen's was edited. The fork's diff is additions only — one skill folder, one bank folder, one README section.

1. Installed the skill as a sibling of his. `.cursor/skills/circuit-bank/` sits beside his `.cursor/skills/problem-creation/` — the same convention he uses.
   → the skill folder in the fork
2. Called his script in place instead of copying it. The forked scripts resolve `../problem-creation/scripts/scenario_list.py`, so the loop that runs is literally his file.
   → his loop script, unchanged

3. Pointed at your circuit skill wherever it lives — a sibling folder, `~/.claude/skills`, `~/.cursor/skills`, or `CIRCUIT_SKILL=`. (The one thing that broke on the first fork run.)
   → circuit-diagram-skill
4. Used his venv and his folder layout. One `.venv` at the repo root; the bank under `PHY II Electricity and Magnetism/1_DC Circuits/`, next to his `PHY I Mechanics/`.
   → the bank folder in the fork

The proof it worked as desired: his own `validate_input.py` accepted the circuit input; his `scenario_list.py` ran the loop (target 30 → needed 30 → complete); the 30-item bank came out identical to the standalone run, because everything downstream of his loop is deterministic.

The files (in the fork): PHY2-DC-SPRTC3-10232026.yaml · PHY2-DC-SPRTC3-10232026.pdf · the 10-item bank (private repo — opens for you)

Three sentences to say to anyone

> “Same difficulty, different solution, every time — that's what isomorphic means.”
>
> “I kept Chen's loop; the rules propose and check the circuits, and the LLM only steps in when the rules run out — and says so.”
>
> “Every number in the bank is calculated, including the wrong answers.”

More: 13-page report · 12-slide deck · 2-slide deck · speaker script · flow chart · why 34